VISION CRAFT

JavaScript Image Processing Codes

Chitranshi Gupta (24155463) | Arpita Samantray (24155912)

1. Grayscale Conversion

```
function convertToGrayscale(imageData) {
    const data = imageData.data;
    for (let i = 0; i < data.length; i += 4) {
        const gray = 0.299 * data[i] + 0.587 * data[i + 1] + 0.114 * data[i + 2];
        data[i] = data[i + 1] = data[i + 2] = gray;
    }
    return imageData;
}
```

2. Negative Transformation

```
function applyNegative(imageData) {
    const data = imageData.data;
    for (let i = 0; i < data.length; i += 4) {
        data[i] = 255 - data[i];
        data[i + 1] = 255 - data[i + 1];
        data[i + 2] = 255 - data[i + 2];
    }
    return imageData;
}
```

3. Sobel Edge Detection

```
function applyEdgeDetection(imageData) {
    const { data, width, height } = imageData;
    const output = new Uint8ClampedArray(data.length);
    const gx = [[-1,0,1],[-2,0,2],[-1,0,1]];
    const gy = [[-1,-2,-1],[0,0,0],[1,2,1]];
    output.set(data);

    for (let y = 1; y < height - 1; y++) {
        for (let x = 1; x < width - 1; x++) {
            let sumX = 0, sumY = 0;
            for (let ky = -1; ky <= 1; ky++) {
                for (let kx = -1; kx <= 1; kx++) {
                    const i = ((y + ky) * width + (x + kx)) * 4;
                    const gray = 0.299 * data[i] + 0.587 * data[i+1] + 0.114 * data[i+2];
                    sumX += gray * gx[ky+1][kx+1];
                    sumY += gray * gy[ky+1][kx+1];
                }
            }
            const magnitude = Math.min(255, Math.sqrt(sumX*sumX + sumY*sumY));
            const i = (y * width + x) * 4;
            output[i] = output[i+1] = output[i+2] = magnitude;
            output[i+3] = data[i+3];
        }
    }
    return new ImageData(output, width, height);
}
```

4. Histogram Calculation

```
function calculateHistogram(imageData) {
    const data = imageData.data;
    const histogram = new Array(256).fill(0);
    for (let i = 0; i < data.length; i += 4) {
        const gray = Math.round(
            0.299 * data[i] + 0.587 * data[i+1] + 0.114 * data[i+2]
        );
        histogram[gray]++;
    }
    return histogram;
}
```

5. Gaussian Blur

```
function applyGaussianBlur(imageData) {
    const { data, width, height } = imageData;
    const output = new Uint8ClampedArray(data.length);
    const kernel = [
        [1,4,6,4,1], [4,16,24,16,4], [6,24,36,24,6],
        [4,16,24,16,4], [1,4,6,4,1]
    ];
    output.set(data);

    for (let y = 2; y < height - 2; y++) {
        for (let x = 2; x < width - 2; x++) {
            let r = 0, g = 0, b = 0;
            for (let ky = -2; ky <= 2; ky++) {
                for (let kx = -2; kx <= 2; kx++) {
                    const i = ((y+ky)*width + (x+kx))*4;
                    const w = kernel[ky+2][kx+2];
                    r += data[i]*w; g += data[i+1]*w; b += data[i+2]*w;
                }
            }
            const i = (y*width + x)*4;
            output[i] = r/256; output[i+1] = g/256;
            output[i+2] = b/256; output[i+3] = data[i+3];
        }
    }
    return new ImageData(output, width, height);
}
```

6. Image Sharpening

```
function applySharpen(imageData) {
    const { data, width, height } = imageData;
    const output = new Uint8ClampedArray(data.length);
    const kernel = [[0,-1,0],[-1,5,-1],[0,-1,0]];
    output.set(data);

    for (let y = 1; y < height - 1; y++) {
        for (let x = 1; x < width - 1; x++) {
            let r = 0, g = 0, b = 0;
            for (let ky = -1; ky <= 1; ky++) {
                for (let kx = -1; kx <= 1; kx++) {
                    const i = ((y+ky)*width + (x+kx))*4;
                    const w = kernel[ky+1][kx+1];
                    r += data[i]*w; g += data[i+1]*w; b += data[i+2]*w;
                }
            }
            const i = (y*width + x)*4;
            output[i] = Math.max(0, Math.min(255, r));
            output[i+1] = Math.max(0, Math.min(255, g));
            output[i+2] = Math.max(0, Math.min(255, b));
            output[i+3] = data[i+3];
        }
    }
    return new ImageData(output, width, height);
}
```

7. Thresholding

```
function applyThreshold(imageData, threshold = 128) {
    const data = imageData.data;
    for (let i = 0; i < data.length; i += 4) {
        const gray = 0.299 * data[i] + 0.587 * data[i+1] + 0.114 * data[i+2];
        const value = gray > threshold ? 255 : 0;
        data[i] = data[i+1] = data[i+2] = value;
    }
    return imageData;
}
```

8. Sepia Transformation

```
function applySepia(imageData) {
    const data = imageData.data;
    for (let i = 0; i < data.length; i += 4) {
        const r = data[i], g = data[i+1], b = data[i+2];
        data[i] = Math.min(255, 0.393*r + 0.769*g + 0.189*b);
        data[i+1] = Math.min(255, 0.349*r + 0.686*g + 0.168*b);
        data[i+2] = Math.min(255, 0.272*r + 0.534*g + 0.131*b);
    }
    return imageData;
}
```